Industrialisation IA

Zacharie BUISSON - Mario DEFRANCE

Introduction

Agenda

Séance 1 - DS - Mario DEFRANCE / 06 octobre 2026

- Cours Data science
- Début des travaux pratiques 1

Séance 2 - DS - Zacharie BUISSON / 09 octobre 2026

- Travaux pratiques 1

Séance 1 - IA - Zacharie BUISSON / 20 octobre 2026

- Cours Industrialisation IA
- Début des Travaux pratiques 2



Séance 3 - DS - Mario DEFRANCE / 23 octobre 2026

- Travaux pratiques 1

Séance 4 - DS - Zacharie BUISSON & Mario DEFRANCE / 10 novembre 2026

- Finalisation travaux pratiques 2
- Soutenance des projets Data Science

Séance 2 - IA - Mario DEFRANCE / 13 novembre 2026

- Travaux Pratiques 2.

Séance 3 - IA - Zacharie BUISSON & Mario DEFRANCE / 08 décembre 2026

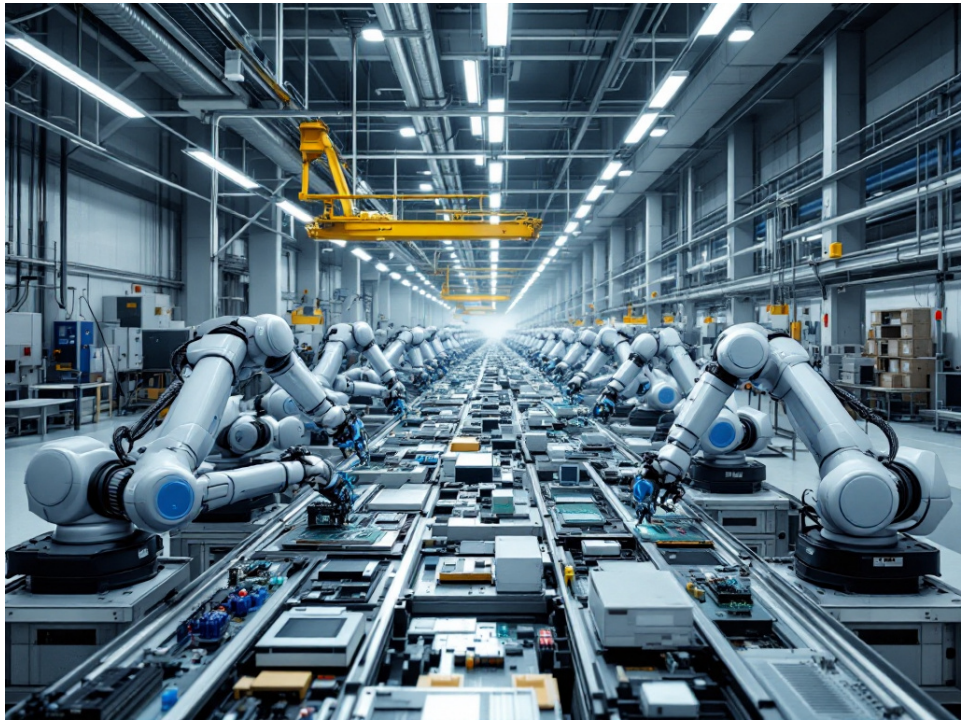
- Finalisation travaux pratiques 2
- Soutenance des projets industrialisation Modèle IA.

Tour de table - Présentation & vos attendus sur le cours



Contexte

Comment valoriser le travail des Data Scientist - ML Engineer



- Vous avez construit (ou êtes en train de construire) votre modèle de Machine Learning / IA
  - ➢ **Il ne sert à rien !**

Pour le valoriser, vous allez :

- le rendre disponible aux utilisateurs à travers une API
- mettre en place une chaîne d'industrialisation automatisée

Une course effrénée à la livraison de nouvelles fonctionnalités



Livraison d'un projet IA : 2 mois à 3 ans

Pour une même fonctionnalité, plus le temps de livraison est long:

- Plus les utilisateurs se tourneront vers un service concurrent :
  - Exemple : ChatGPT interne vs ChatGPT externe
    - ➢ Problèmes de sécurité des données
    - ➢ Perte de clients
    - ➢ Déception des collègues qui préfèrent se tourner vers des prestataires externes, start-ups…

- Plus il faut payer le data scientist longtemps…
- … plus le ROI du projet diminue
- Plus il faudra de temps pour itérer et corriger les erreurs en production et répondre aux nouveaux besoins métiers…

➢ Nécessité d'automatiser l'industrialisation pour livrer le plus vite possible

Réconcilier les visions : Data Scientist vs Métier vs Utilisateur

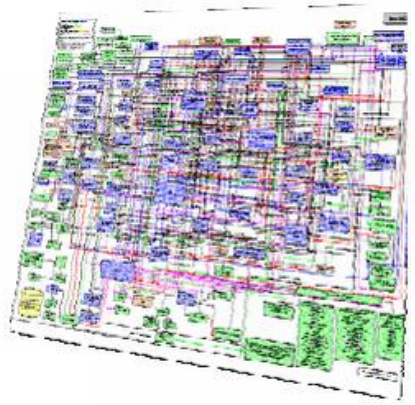


L'intérêt de la solution reste à démontrer en production !

- Besoin de monitoring des performances
  - Est-ce que la solution est utilisée ?
  - Est-ce que la solution fonctionne ?
  - Est-ce que la solution est performante ?

- Besoin de feedback utilisateurs (pas forcément évident en IA…)
  - Biais de sélection: comment savoir si un prospect qui n'a pas pris un prêt chez nous l'aurait payé ?
  - Comment savoir si un utilisateur à utiliser la prédiction de l'IA ? sans ajouter de surcharge ?

Réconcilier les visions : Data Scientist vs Système d'information



Vous n'êtes pas non plus les seuls développeurs dans l'entreprise :

Dans une entreprise, il y a des personnes chargées de la production : ce sont les personnes du SI (système d'information) ;

Ils gèrent les outils informatiques au niveau de l'entreprise :

- Site WEB de l'entreprise
- Outils pour les commerciaux
- Dashboards…

Votre modèle d'IA doit s'intégrer dans cet environnement complexe :

- un héritage technique existe et les solutions modernes doivent s'intégrer
- des contraintes pour rendre le système homogène doivent être mises en place (sécurité, qualité de code..)

Infrastructure d’entreprise

Choisis ton environnement - Cloud VS On Premise



Choix 1 : Architecture Cloud

Une architecture distante

Avantages

- *Utilisation à la demande (pay what you use)*
- *Services pré existants (délégation software)*
- *Pas de serveurs (délégation hardware)*

Inconvénients

- *Dépendance à services externes : obligation de montée de version, suppression de service*
- *Protection des données, notamment extra UE*

Azure (Microsoft)

AWS (Amazon)

OVH



Choix 2 : Architecture on premise

Une architecture à la maison

Avantages

- *Flexibilité hardware & software : sur mesure aux besoins*
- *Indépendance aux organismes externes*
- *Sécurité gérée en interne, pas de problèmes de transit de données*

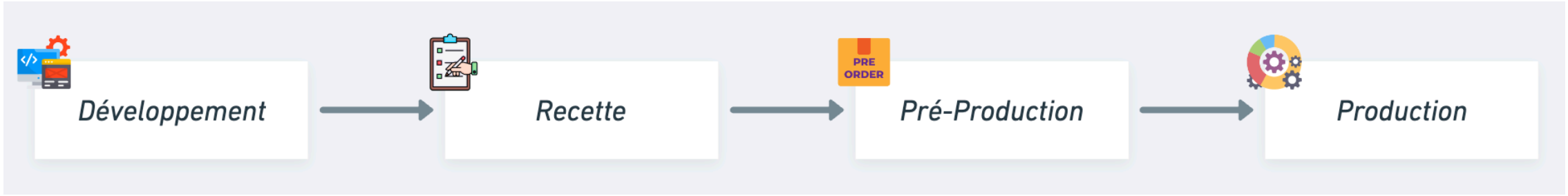
Inconvénients

- *Mise à jour hardware et software à la main de l'entreprise : ajout de puissance de calcul manuel…*
- *Recrutement de personnel pour maintenir le système*

Docker

Kubernetes

Le cycle de vie d'un projet IT en production



Développement	Recette	Pré-Production	Production
Data Scientist	*Data Scientist, Dev Ops*	*Equipe Infrastructure (App Ops, Dev Ops)*	*Equipe Infrastructure (App Ops, Dev Ops)*
Livraison d'une première version dans un environnement industriel	Livraison de l'ensemble des briques composants le système final	Verifier que l'infrastructure répond correctement	Mise à disposition des utilisateurs
Instable	**Moyennement stable**	**Stable**	**Stable**
Tests d'intégration Tests de masse	Tests de masse Tests fonctionnel	Tests de charge Tests d'intrusion Tests fonctionnels	Tests d'intrusion Smoke tests

A propos des données - stockage, utilisation…

Bases de données relationnelles

Modèle historique dans les banques et assurances

SQL : Structured Query Language

Approche structurée

- Système polyvalent - rapide en lecture et écriture ;
- Pas d'unicité dans les observations ;
- Optimisé pour le brassage de données : jointure, calculs de variables en fonction de caractéristiques d'intérêt.

Oracle DataBase

PostgreSQL

NoSQL : Not only Structured Query Language

Approche non structurée

- Optimiser en lecture - pas écriture ;
- Accès aux données par clé (rapide), unicité des clés ;
- Utilisation sur de la donnée brute, en production.

Microsoft SQL Server

mongoDB

Bases de données non relationnelles

Modèle novateur dans les banques et assurances

- Stockage documentaire : note de procédure interne, contrats client…
- Vidéo : caméra de surveillance, pièces de sinistres ;
- Voix : call center.

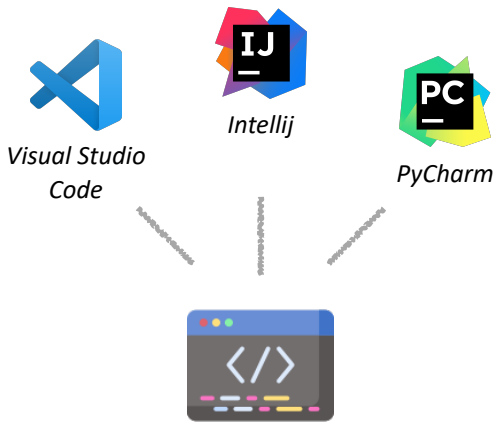
GED : Gestion Electronique de Documents

Amazon S3

Microsoft Azure Blob Storage

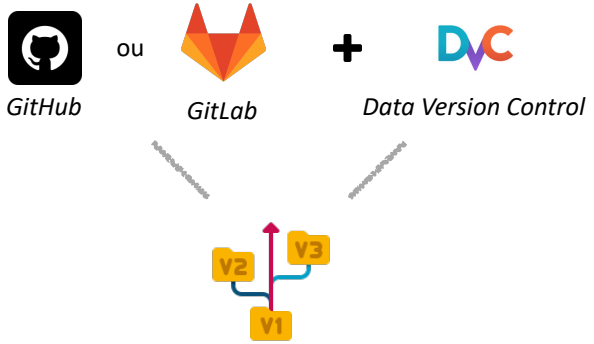
Programmation, bonnes pratiques et comment construire un logiciel utile

Quelques étapes pour la réussite d'un projet IA en collaboration



Utilisation d'un IDE

But : Coder avec des avantages



Versionner son code et ses données

But : Collaborer et retracer l'histoire de ceux-ci



Définition d'un environnement de travail

But : Avoir les mêmes fonctionnalités

Poetry ou *UV* + *PyEnv*



Tracer ses expérimentations de ML

But : Historiser les résultats des tests

mlflow

Bonne pratique de programmation Python : PEP8, Orienté Objet et SOLID

PEP8 : Python Enhancement Proposal

Norme de programmation Python

Ensemble de pratiques pour faire du code dans les standards Python

- Respecter la snake_case ;
- Faire des doctrines ;
- Ne pas laisser d'éléments non utilisés…

But : Normalisation du code, possibilité de le rendre compréhensible de tous…

Programmation Orientée Objet

Synthèse de fonctions pour suivre dans le temps



Exemple d'object : Client de Banque

Attributs : Montant sur le compte, nom, prénom

Objet hérité : Client crédit agricole

Attributs : Nom de l'agence, caisse régionale…

But : Regrouper et normaliser des éléments avec des caractéristiques communes, suivre l'état d'un élément…

Programmation SOLID

Comment construire une bonne fonction ?

Single Responsability Principle

Chaque fonction doit avoir une seule fonctionnalité

Open / Closed Principle

Ouvert à l'extension, fermé à la modification

Liskov Substitution Principle

Les classes héritées doivent avoir les méthodes de la classe mère fonctionnelles

Interface segregation principle

Une classe ne doit avoir que des fonctions utiles à sa classe

Dependency Inversion Principle

Non détaillé, pas le plus important…

But : Avoir des fonctions simples à tester, limiter les erreurs…

3 principaux modes de livraison de code



Packaging de code

Mise à disposition du code en ligne



Comment faire ?

Code brut livré dans un repertoire git
Construction d'une Wheel
Déploiement de la wheel en ligne :

- *PyPI*
- *Conda Forge*
- *Artifactory interne*



But : Utilisation et installation d'un code à la demande et à la main d'un utilisateur.



Batching & CronJob

Publier une application à lancement auto.



Comment faire ?

Code brut livré dans un repertoire git
Encapsulation dans un outil

- *CronJob*
- *Lambda + EventBridge sur AWS*
- *DataBricks*
- *Linux : Cron*
- *Windows : Task Scheduler*



But : Déploiement / installation en production, lancement automatisé chaque jour, à intervalle régulière, envoi notification succès / échec.



API

Publier une application en temps réel



Comment faire ?

Code brut livré dans un repertoire git
Construction d'une API
Encapsulation dans un conteneur Docker
Déploiement du docker sur un serveur

- *ArgoCD (on premise)*
- *AWS ECS (cloud)*
- *Azure Machine Learning*



But : Déploiement / installation en production, application disponible pour un appel en temps réel.

Zoom sur le fonctionnement d'une API

Comment Interagir avec une API ?

API et requêtes..

Quatre types de requêtes principales :

GET

Sert à récupérer des informations directement

POST

Sert à envoyer une information (sensible) à l'application

PUT

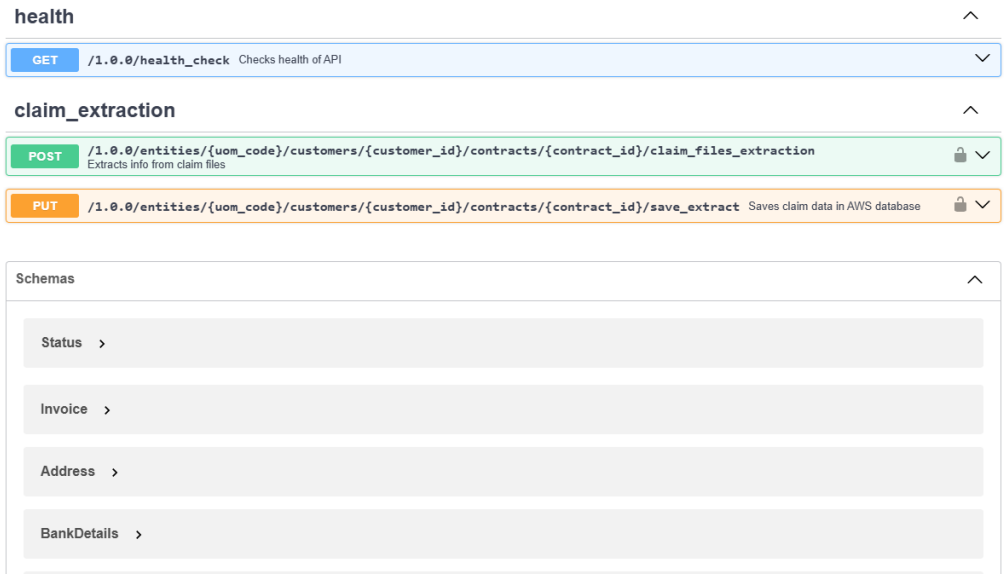
Enregistrer une donnée dans une BDD ou la mettre à jour

DELETE

Supprimer une donnée dans une base de données

Documentation : OpenAPI (swagger)

Fichiers .yaml, ou .json qui décrit les façons possibles pour interagir avec un API



Comment livrer un logiciel

Le code fonctionne sur mon ordinateur mais pas en production que faire ?



Créer un plat avec une recette de cuisine

Ecrire la recette de cuisine
Décrire les étapes permettant de créer la recette.

Faire la recette de cuisine
Exécuter les étapes une à une pour construire un plat.

Avoir un plat préparé
Avoir un plat à disposition, prêt à être utilisé.



Créer une application avec Docker

Ecrire la recette de l'application
Décrire les étapes permettant de reconstruire l'application (DockerFile)

Commande : pas de commande
Il faut juste rédiger un DockerFile

Faire la recette de l'application
Exécuter les étapes une à une pour construire l'application. (Image Docker)

Commande : docker build
Peut être une image stockée en local ou dans un repertoire en ligne : artifactory, Docker Hub, AWS ECR.

Avoir une application préparée
Avoir une application déployée, dans un espace isolé et performant. (Conteneur)

Commande : docker run
Peut s'utiliser en local ou sur un serveur avec Kubernetes

J’ai 500M d’utilisateurs de plus ce matin, ça passe non ?

Points Critiques à surveiller :



Gestion de la charge de l’application

Comment gérer un afflux d’utilisateurs ?



Redémarrer l’application si échec

Cela permet de maintenir une application fonctionnelle



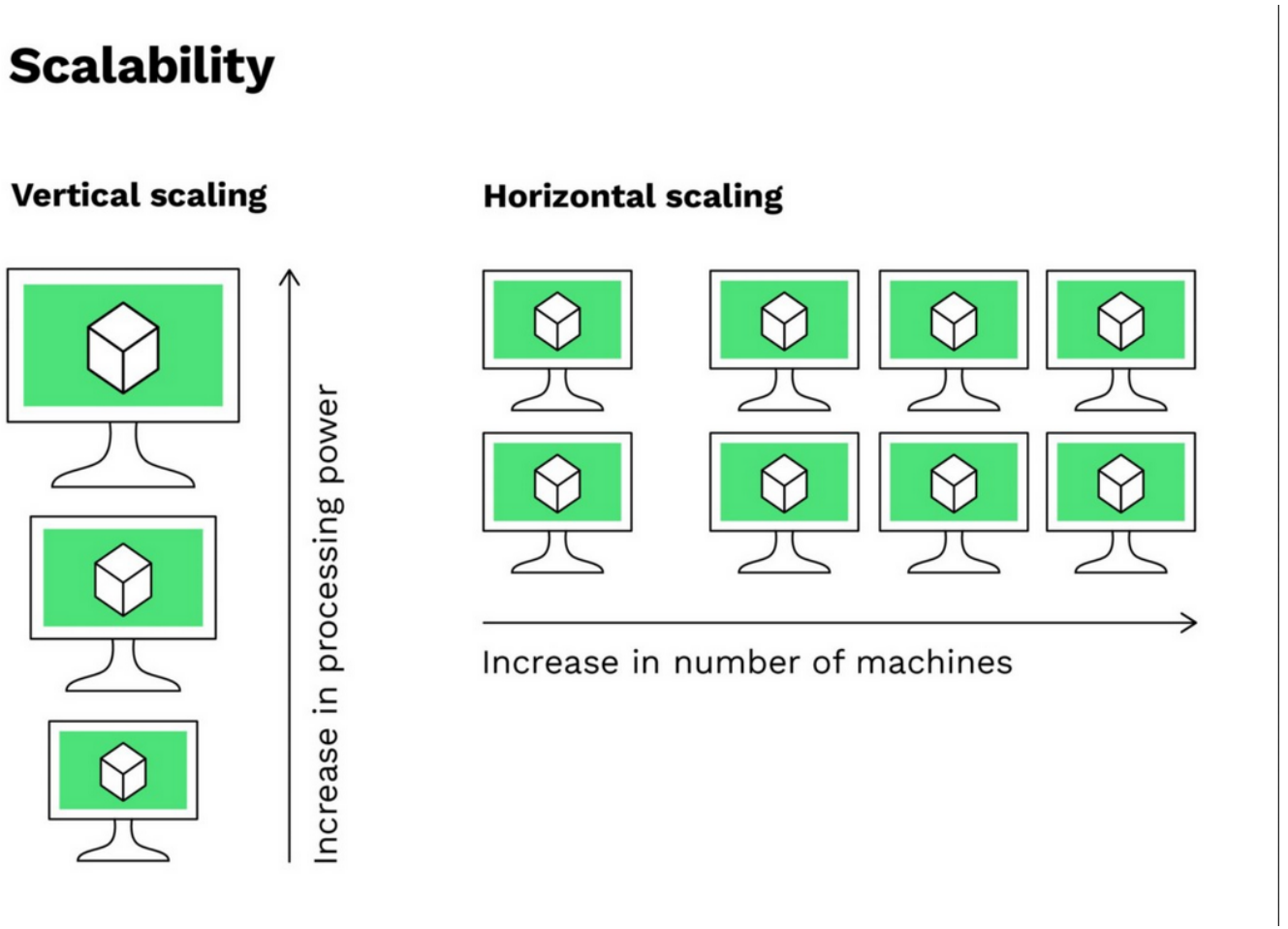
Gérer les mises à jour

Comment mettre à jour une application sans tout couper ?



Tester mon application

Comment s’assurer que mon application soit correcte ?



Intégration Continue : CI

Intégration continue

Comment intégrer efficacement des modifications dans une application de production ?

Evaluation de la qualité de code

Solutions Open Source

- *Flake8 : Formateur de code*
- *Black : Formateur de code*
- *Ruff : Formateur de code (rapide)*
- *PyLint : Formateur de code*
- *Isort : Formateur des imports*

sonarqube

But : S'assurer que le code de l'application sera compris par tous.

Tests unitaires

- Vérifier que chaque fonction de son code marche correctement
- Tester sur des cas classiques et des cas limites.

pytest

unittest

But : S'assurer que le code fonctionne et que l'on ne « casse » rien qui fonctionnait.

Construction des artefacts

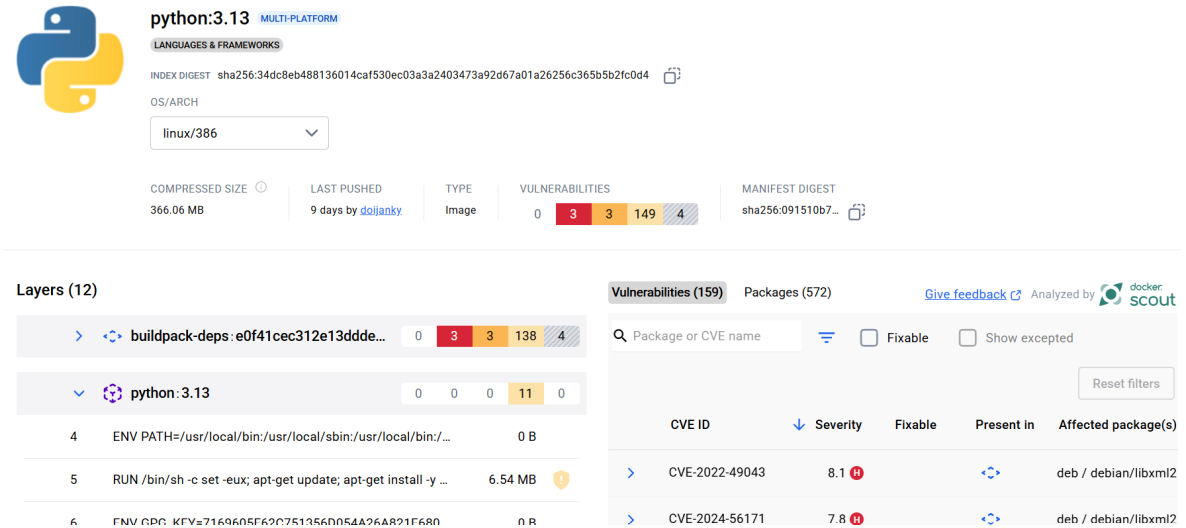
- Construction de l'image contenant l'API ou d'un .zip ;
- Stockage de l'artefact créé dans un répertoire.

DOCKER

But : Faciliter et normaliser la livraison du code.

Déploiement/Livraison en Continue : CD

1. **tests de sécurité Xray CVE : vulnérabilités de l'application + test d'injection de code dans les**
2. **déploiement de l'application dans un environnement**
3. **promotion vers environnements ultérieurs de DEV à PROD**





Application à l’IA

MLOps : suivre la viabilité d'une application IA



Suivi et monitoring de la viabilité de l'application en production

Ne pas laisser son application dépérir en production…



Suivre le comportement de son application :

Est-ce que l'application fonctionne ?

Fait-elle face à des imprévus ? Est-elle utilisée ?

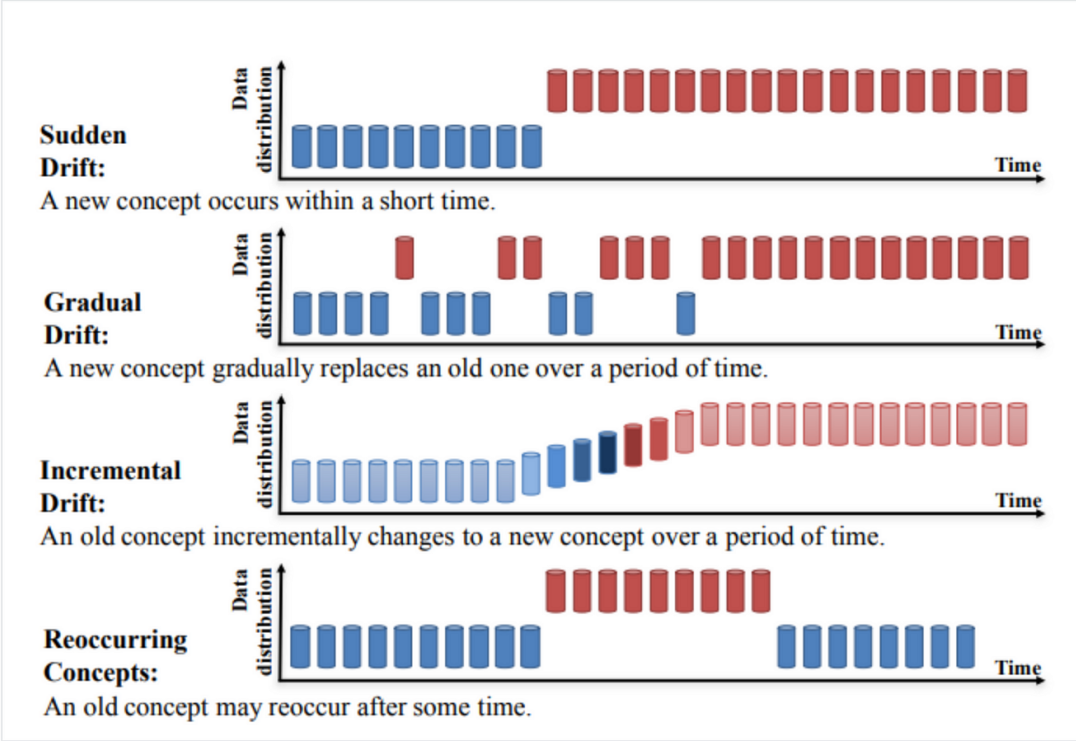
S'est-elle correctement intégrer dans le reste du SI ?



MLOps : suivre les performances de l'IA

Suivi et monitoring du drift en production

Ne pas laisser son application dépérir en production…



Source: *https://arxiv.org/pdf/2004.05785.pdf*

Recueillir des retours des utilisateurs :

Pas possible : un fraudeur ne va pas faire un feedback
A prendre en compte dès la conception du projet





Surveiller la cohérence statistique d'un modèle d'IA / ML :

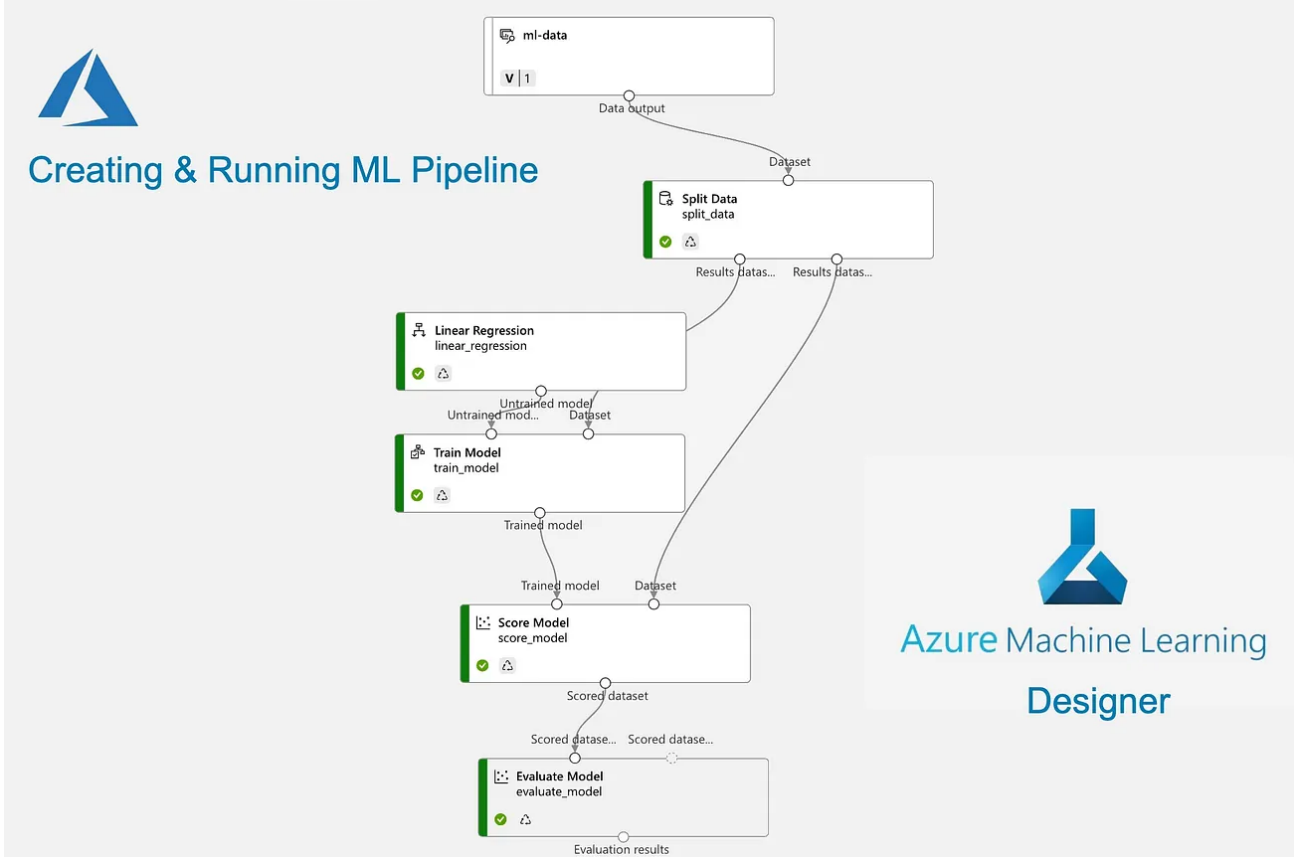
Attention aux données plus représentatives de la réalité
Nécessite un ré-entrainement ou des correctifs.

MLOps : pipeline d'IA & Entrainement automatique

Pipelines d'IA

Entrainement automatique de modèles

- Script d'entraînement d'un modèle d'IA, prévu de A à Z.
- Peut être déclenché automatiquement ou à dates régulières
- Permet d'itérer rapidement et de créer de nouveaux cas d'usage similaire



Script d'entrainement

Création de blocs thématiques réutilisables au besoin